Winning Is Not Killing: Active Win Rate and Expert Iteration with a Network-Free Kill-Search in Bomberman

Anonymous Author(s)
Affiliation withheld for review

Abstract—Bomberman agents are usually ranked by win rate, although a game can be won because the opponent destroys itself or is caught by a shrinking arena. We define the active win rate, the share of games won with at least one kill credited to the agent's own bomb, and use it to study how a hand-written kill-search without a neural network can be turned into a fast neural policy. Against a fixed rule-based bot, policy-gradient training with a safety mask reaches 40 to 51 percent active win rate, and over a dozen standard refinements fail a preset gate. A search that looks up to three moves ahead lifts the same network to 94 percent. As a sparse loss inside reinforcement learning the search does not help; dense distillation of its labels does. Three rounds of expert iteration produce a convolutional policy at 92 percent, within one point of its search teacher, with one network evaluation per move. Interventions that raise the win rate by up to 16 points leave the active win rate unchanged. Against frozen neural opponents most games are drawn, and with the correct opponent model a search finds a kill plan in only 16 percent of them.

Index Terms—Bomberman, evaluation metrics, expert iteration, policy distillation, reinforcement learning

I. Introduction

Deep reinforcement learning (RL) reaches human or grandmaster level in Atari and StarCraft II [1], [2], but with compute far beyond a single consumer machine. Bomberman, and its research variant Pommerman [3], is a hard test for model-free RL at small scale: a bomb placed at random kills its owner about 40% of the time [4], the reward for a kill arrives many steps after the decision that made it, and decisive moments are rare. Large RL efforts needed hundreds of CPUs [5] or long population-based training [6] and still lost to search-based agents [6], [7].

A second problem is how agents are scored. Several groups noticed that an agent can collect wins when the opponent kills itself [6], [8]–[10], but results are still reported as win rate (WR). An early checkpoint of ours won 52.5% of 40 four-player games while killing nobody with its own bombs.

This paper reports a controlled study, run on consumer hardware, of how to get a fast neural Bomberman agent that wins by killing. The contributions are:

- C1, a metric. ActiveWR, the probability of winning and killing at least one opponent with one's own bomb. WR overstates three of our interventions by 3.75–16 pp.
- C2, three uses of one search. Sparse search labels as an auxiliary RL loss, search at play time and dense distillation of search labels: only the last two help (-2.75, $+16.5$, $+16.25$ pp), testing two directions proposed by Kartal et al. [9].
- C3, expert iteration with a network-free expert. Three rounds of "student plus deeper search → distil" raise a plain CNN from 48.25% to 92.25% ActiveWR; the student–teacher gap shrinks from -3.75 to -0.75 pp.
- C4, controlled negative results for GAE, PPO clipping, advantage normalisation, reward changes, sudden-death training, group-relative advantages and KL-anchored RL after distillation.
- C5, a draw analysis. With the correct opponent model, a search finds a kill plan in only 15.9% of the draws between strong agents.

Each building block is known; the contribution is the metric, the controlled comparison under a fixed protocol, and the analysis of where the remaining draws come from. Fig. 1 summarises the path to the final agent.

II. Related Work

RL in Bomberman and Pommerman. Without action filtering PPO learned nothing in five days; with a filter it learned [4]. Skynet combined PPO, an action filter and reward shaping, and observed that wins often came from opponents killing themselves [8]. Imitation then RL forgot catastrophically [11]; curriculum and population-based self-play still lost 70–95% of games to search agents after 200 million steps [6]. Backplay [12] and continual match training on 220 CPUs [5] attacked the sparse reward.

Search in Pommerman. The NeurIPS 2018 winner used depth-limited tree search with pessimistic opponent scenarios [7]. Statistical forward planning produced 59–84% draws between strong planners [13]; MCTS with learned opponent models can turn defensive [10].

Search as a teacher. UCT decisions trained real-time Atari CNNs [14]. Expert iteration (ExIt) [15] and AlphaZero [16], [17] alternate a search expert and a network apprentice, with the network guiding the search. In Pommerman, MCTS was a demonstrator inside A3C on an 8×8 board [9], [18]. Distillation [19], [20], DAgger [21] and kickstarting [22] give the supervised side of our method.

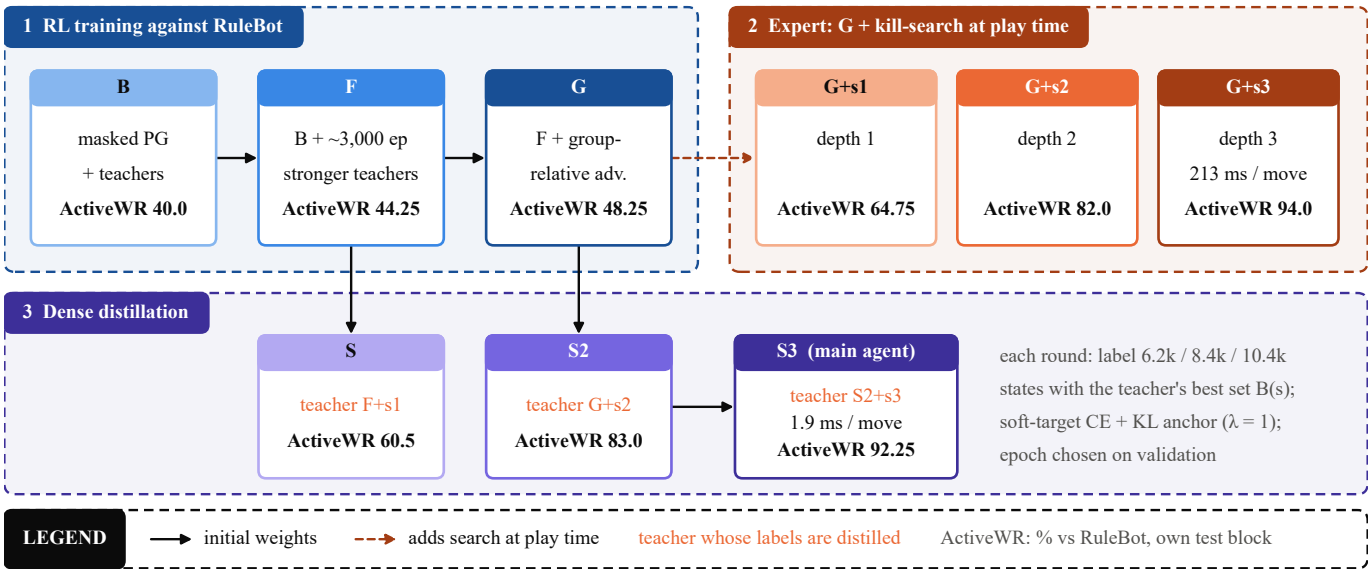


Fig. 1. Path from the first RL checkpoint to the final agent. Panel 1: RL-trained CNNs. Panel 2: G with kill-search at play time (the expert); the dashed arrow adds search without changing weights. Panel 3: distilled CNN students; black arrows give the initial weights, the orange label names the teacher whose labels are distilled. Numbers are ActiveWR (%) against RuleBot, each on the model's own single-use test block (Table I).

Safety and evaluation. Masking invalid actions in the logits keeps a valid policy gradient [23]; only potential-based shaping preserves optimal policies [24]. RL results need confidence intervals over independent units [25], [26]; we use the bootstrap [27] over paired boards.

III. Problem Setting

Game. A NumPy re-implementation of Pommerman-style Bomberman on a 9×9 board with a border, rigid pillars on (even, even) cells and 70% wooden walls elsewhere except 3×3 spawn zones. Two agents start in opposite corners; actions are STOP, four moves and BOMB. A bomb explodes after 8 steps into a cross of range 2, destroys wooden walls (a power-up appears with probability 0.3), kills agents in the blast and chains into other bombs. A game ends when at most one agent is alive or after 500 steps (draw). Our default sudden-death schedule E closes the arena from the border inwards from step 475 every 25 steps, with a 40-step warning.

Observation. Each agent sees 13 9×9 planes: walls, bomb timers, flames, two power-up types, both positions, free bomb slots, capacity, range, the inverse time until each cell is hit by a forecast blast including chains, and the ring warning.

Safety mask and opponent. Every agent acts through the same safe-action mask $m(s) \in \{0,1\}^6$, which removes moves into cells a forecast blast reaches before the agent can leave and bombs without an escape route. The opponent is a rule-based bot (RuleBot) that dodges, collects items, attacks with the correct blast range and uses the same mask; when no rule applies it picks a random safe move, so its play is stochastic. RuleBot is fixed throughout.

Metric. Let W be the event that the learner is the last agent alive and K that at least one opponent died in a blast credited to a learner bomb (credit follows the owner of the bomb that started a chain). We report $\mathrm{ActiveWR}(\pi) = \Pr_\pi[W \wedge K]$ and decompose every evaluation into active, passive (opponent self-kill) and ring wins, draws, self-kills and other losses. $\mathrm{WR} = \Pr[W]$ is secondary.

IV. Method

All agents share one actor–critic CNN: three 3×3 convolutions (32, 64, 64 channels, padding 1), a 256-unit layer, a 6-way actor head and a critic head (1.39M parameters). Logits of unsafe actions are set to -10^9 before the softmax, which keeps the policy gradient valid [23]. Actions are sampled: argmax play lowered F's ActiveWR by 9.25 pp [-16.0, -2.5] and raised draws by 7.5 pp on validation.

A. Stage 1: masked policy gradient with auxiliary teachers

The RL stage trains against RuleBot with Monte-Carlo returns ($\gamma = 0.99$) and a learned baseline (REINFORCE with baseline [28]; GAE [29] and PPO clipping [30] were tested as variants), an entropy bonus and cross-entropy terms towards four rule-based teachers (escape, combat, navigation, ring safety; weights 0.5, 0.9, 0.4, 0.6). Rewards are event-based (+4 per credited kill, −1 on death, −1.5 extra for a self-kill, small item and wall bonuses); the +10 for being last alive is kept only for active wins. A game of about 230 steps has only about 2.2 steps where the actions' search scores differ and 1.25 where some action kills, so we also tried a GRPO-like update [31] at such decisive states: the policy is replayed $K = 16$ times for 20

Algorithm 1 Kill-search of depth d (play time)

Require: state s, mask m, CNN action a_0, depth d, horizon $H = 12$
if no opponent within distance 6 then return a_0
end if
for a with $m_a(s) = 1$ do
    $v_1(a) \leftarrow \text{Rollout}(s, [a], H)$ ▷ RuleBot opponent
end for
if $\max_a v_1(a) > v_1(a_0)$ then return $\arg\max_a v_1(a)$
end if
for $\ell = 2, \ldots, d$ do ▷ only if no shorter kill
    for safe prefixes $(a, b_2, \ldots, b_\ell)$ do
        $v_\ell(a) \leftarrow \max \text{Rollout}(s, (a, b_2, \ldots, b_\ell), H)$
    end for
    if some $v_\ell(a) > 0$ and $v_\ell(a_0) \leq 0$ then return first move of a kill plan
    end if
end for
return a_0

ticks, with outcome $o_k \in \{+1, -1, 0\}$ (credited kill, death, neither) and advantage $o_k - \bar{o}$. The resulting checkpoint G is our best RL policy.

B. Stage 2: network-free kill-search

For each legal first action, a copy of the game is simulated for 12 ticks (fuse 8 + flame 2 + slack) with the opponent playing RuleBot and the learner only dodging afterwards (Algorithm 1). A rollout scores +1 if an opponent dies from a learner bomb while the learner lives, −1 if the learner dies, else 0. Depth 2 or 3 adds learner moves before dodging and is expanded only when no shorter plan kills. The search is skipped unless an opponent is within Manhattan distance 6, and the CNN's sampled action is kept unless another action scores strictly higher.

C. Stage 3: dense distillation and expert iteration

The teacher (current student + depth-d search) plays 3,000 games against RuleBot on fresh boards. At every learner step the search scores all legal actions; a decision state is one where the scores differ, and its target spreads mass uniformly over the best set $B(s) = \arg\max_{a:m_a(s)=1} v(a)$. With probability 0.05 an ordinary state is stored as an anchor with the starting network's logits. The student starts from the previous network and minimises

$$\mathcal{L} = \mathbb{E}_{\mathcal{D}_{\text{dec}}}\Big[-\textstyle\sum_a q(a \mid s) \log \pi_\theta(a \mid s)\Big] + \lambda\, \mathbb{E}_{\mathcal{D}_{\text{anc}}} \mathrm{KL}\big(\pi_{\text{ref}} \| \pi_\theta\big) \tag{1}$$

with $\lambda = 1$, Adam (10^{-4}), batch 256 and 8 or 16 epochs; the epoch is chosen on validation. Round 1 distils F + depth 1 into S, round 2 G + depth 2 into S2, round 3 S2 + depth 3 into S3 (6,171, 8,449 and 10,425 decision states). Unlike ExIt [15], the expert never uses the network's prior or value; the student improves only because the expert also plays the student's own moves where the search is indifferent, keeping the data close to the student's state distribution [21].

V. Evaluation Protocol

Every experiment follows the same pre-specified protocol (fixed in our repository before each run, not in a public registry). (i) Configuration, source hashes and selection rule are written before training. (ii) Checkpoints are selected only on validation seed 21000. (iii) Selected models are tested once on a fresh block of 200 boards, each played from both corners (400 games); a test block is never reused. (iv) Models are compared with a paired bootstrap [27] over boards (10^4 resamples). (v) A change is accepted only if $\hat{\Delta} \geq +5$ pp, the lower CI bound is above zero and self-kills rise by at most 1 pp. One 400-game block of the same pair varies by about ±5 pp across blocks, which is why the gate needs both a margin and a positive bound. RQ1 asks how much WR misjudges agents, RQ2 which use of a search transfers its strength to a CNN, RQ3 how close expert iteration brings a CNN to the search, RQ4 why RL refinements fail, and RQ5 whether draws between strong agents are missed chances.

VI. Results

A. RQ1: win rate misjudges agents

Fig. 3 shows four interventions measured with both metrics on paired boards. A learner-side mask margin of 15 steps for the shrinking ring raised WR by +10.75 pp [6.25, 15.5] but ActiveWR by only +0.75 [-1.25, 2.75]: deaths in the ring fell from 116 to 76 of 400, so the gain was survival. A rule that makes the learner stand on the opponent's escape cell raised WR by +3.75 while ActiveWR fell by 1.75, turning 16 of F's active wins into passive ones. Turning sudden death on adds 16 pp of WR to G with no ActiveWR change. Fig. 4 shows the pattern across all milestones.

B. RQ2: three uses of the same search

Table II compares three ways to use the depth-1 kill-search. As a sparse auxiliary loss inside RL (labels only where the search disagrees with the CNN, about 0.9 per episode, each used once) it did not help. At play time it adds +16.5 pp. Densely distilled (about 6,200 decision states with counter-examples, 8 epochs), it moves the CNN by +16.25 pp with no search at play time. The student learned the condition "only BOMB kills here": the probability of the best move on held-out decision states rose from 0.52 to 0.91, while P(BOMB) on ordinary states stayed at 0.088 → 0.095.

C. RQ3: expert iteration with a network-free expert

Search depth. Fig. 5 links depths on shared test blocks: depth 1 adds +16.5 over G, depth 2 adds +21.0 [17.0, 25.25] over depth 1 and depth 3 adds +8.25 [5.25, 11.5] over depth 2, nearly removing draws (5.75 → 0.25%). The gains come from creating a kill (step, then bomb): depth-1 search plateaus near 65% whatever CNN it wraps.

TABLE I

Main results against RuleBot (9×9, 1v1, schedule E, 400 test games per model, each on its own single-use test block; only rows on the same block are compared inferentially, Fig. 2). CI: 95% bootstrap over boards. *B's original test predates sudden death; the row is B under schedule E on a cross-check block.

Agent	Description	Test block	ActiveWR % [95% CI]	WR %	Passive/ring win %	Draw %	Self-kill %	Mean steps
B*	masked PG + teachers	60000	40.0 [35.0, 45.0]	64.0	24.0	22.5	0.5	357
F	B + ∼3,000 RL episodes	43000	44.25 [39.25, 49.25]	63.5	19.25	25.0	1.0	340
G	F + group-relative advantage	46000	48.25 [43.5, 53.25]	69.0	20.75	23.25	0.75	335
G+s1	G + kill-search, depth 1	46000	64.75 [60.25, 69.25]	84.0	19.25	15.25	0.0	287
G+s2	G + kill-search, depth 2	47000	82.0 [78.25, 85.75]	93.75	11.75	5.75	0.0	208
G+s3	G + kill-search, depth 3	49000	94.0 [91.75, 96.25]	99.5	5.5	0.25	0.0	114
S	distil F+s1 (round 1)	43000	60.5 [55.25, 65.75]	78.5	18.0	18.0	1.0	279
S2	distil G+s2 (round 2)	48000	83.0 [79.5, 86.5]	90.25	7.25	7.0	0.25	211
S3	distil S2+s3 (round 3)	57000	92.25 [89.5, 94.75]	98.0	5.75	0.5	0.25	131

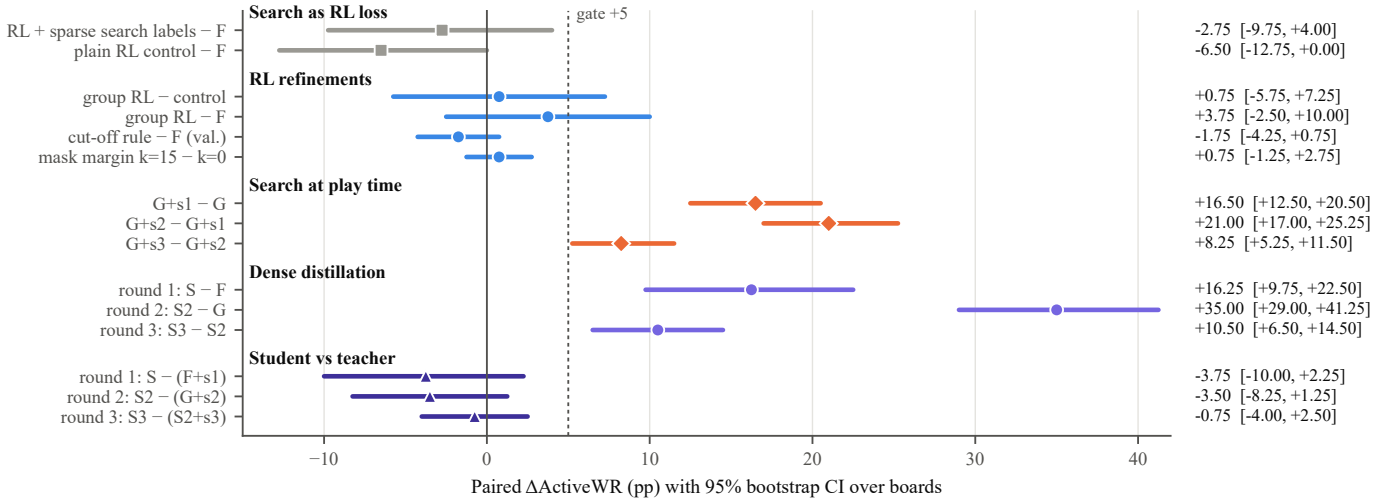


Fig. 2. Paired ΔActiveWR with 95% bootstrap CIs over boards, recomputed from per-game logs. Dashed line: +5 pp gate. "val." marks a validation-block comparison.

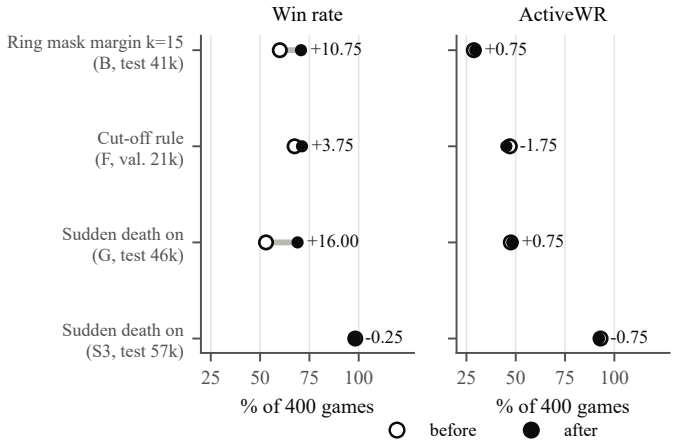


Fig. 3. Win rate versus ActiveWR for the same paired games; labels give after-minus-before (pp).

Iterated distillation. Each round closes most of the student–teacher gap: S − (F+s1) = −3.75 [-10.0, 2.25], S2 − (G+s2) = −3.5 [-8.25, 1.25], S3 − (S2+s3) = −0.75 [-4.0, 2.5] (versus G+s3: −1.75), while the teachers get stronger. Round gains are +16.25, +35.0 [29.0, 41.25] and +10.5 [6.5, 14.5]. The S3 gain is active: passive wins change by −1.5 [-4.5, 1.5], draws by −6.25, games are shorter (131 versus 202 steps) and disabling sudden death gives the same +10.75. Repeating round 3 with new collection boards and train seeds gives S3 − S2 of +10.5, +9.0 and +9.25 on a shared fresh test block (s.d. 0.8); all three pass the gate. Held-out agreement with the teacher's best set rises from 60.5 to 94.0% in round 2 and from 74.8 to 89.8% in round 3, while the KL to the reference policy on anchors stays below 0.11 nats. The main fix is movement, not bombing: G walks the opposite way to the search in up to 67% of states and almost never stops when STOP

TABLE II

Three uses of the same depth-1 kill-search (paired ActiveWR differences, own test blocks).

Use	Comparison	ΔActiveWR [95% CI]	Block
Sparse loss in RL	search arm − F	−2.75 [-9.75, +4.0]	42000
Search at play time	G+s1 − G	+16.5 [+12.5, +20.5]	46000
Dense distillation	S − F	+16.25 [+9.75, +22.5]	43000

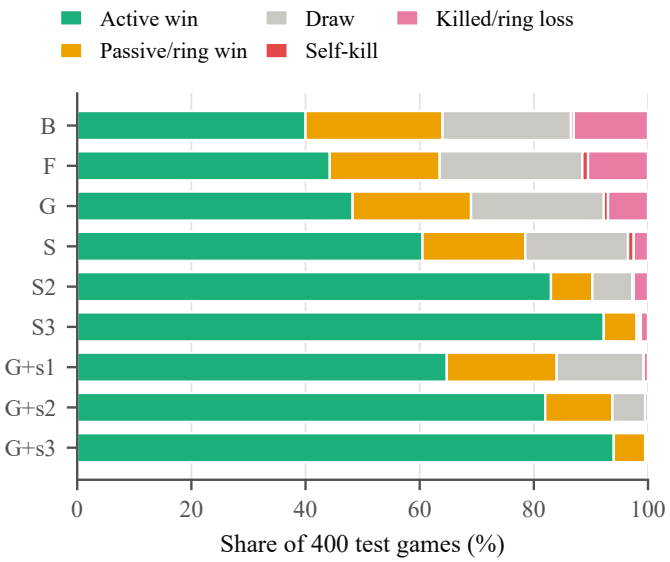


Fig. 4. Outcome composition of the 400-game tests of Table I. RL checkpoints win many games passively or by the ring; students and search agents win by killing.

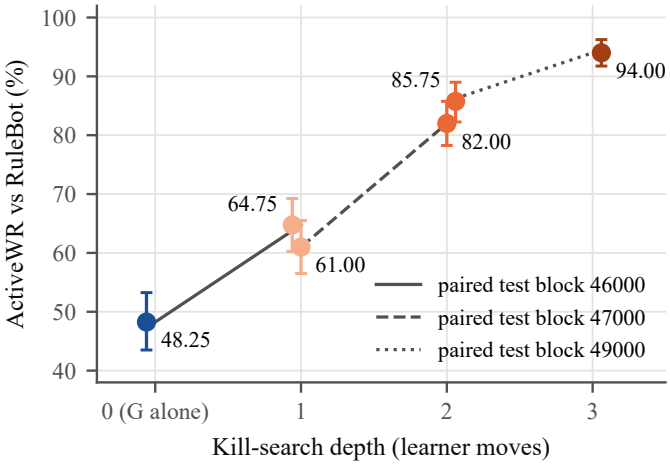


Fig. 5. ActiveWR versus kill-search depth around G; each line style joins two agents tested on one paired block. Error bars: 95% CIs over boards.

is best, while BOMB is already right in 94%.

A stronger base does not make the search stronger: S2+s3 versus G+s3 on validation is −5.0 [-11, 1]. A depth-3 decision costs 213 ms on one CPU thread against 1.9 ms for one S3 forward pass and sample (about 115×).

D. RQ4: ablations and controlled negative results

Table III collects every gated comparison. From B, 200-episode arms with teacher variants, advantage normalisation, GAE, reward changes and PPO clipping stayed within a few points of B, and training under sudden death lowered ActiveWR by 2.25 pp. From F, 3,000 more RL episodes did not help (−6.5), nor did group-relative advantages on decisive states (+0.75 against the matched control), although they raised the kill rate at those states

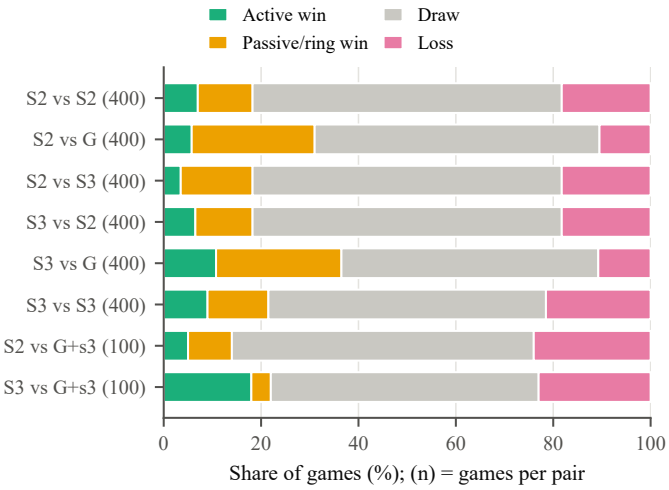


Fig. 6. Outcomes of CNN-versus-CNN and CNN-versus-search games on validation boards (learner first; games per pair in brackets).

by +7.12 pp [4.86, 9.72]. Pushing BOMB at narrow states raised P(kill move) from 0.72 to 0.97 but not kills per game, and no KL weight separated "bomb at the right moment" from "bomb anywhere". Self-play RL from the distilled S2 erased the skill (ActiveWR 83.75 → 65.0 and 57.5), and a global KL anchor kept only 2.5 pp of it.

E. RQ5: draws between strong agents

Against frozen CNNs (S2, G, S3) S3 still draws 53–64% of validation games, mostly at the 500-step limit (Fig. 6); against RuleBot it draws under 1% with a median game of 105 steps. To ask whether these draws hide missed kills, we replayed 195 draws of S3 and ran the depth-3 search at every fifth state with two opponent models (Fig. 7). Under the RuleBot model the search sees a kill plan in 90.3% of draws; with the opponent's own CNN as model, in only 15.9% [9.9, 22.7], while all 20 control active wins contain a plan. Starting the ring earlier did not help either: ActiveWR changed by −1.42 to −0.17 pp, and 80–87% of the remaining draws became simultaneous deaths in the last ring. Both agents use the same safety mask, so a CNN almost always keeps an exit and a forced kill rarely exists.

F. Larger board and free-for-all

Interpolating S3 to 11×11 gives 46.25% ActiveWR; one distillation round on 200 teacher games produced S3-11 with 87.25% on a fresh test block, +41.0 [34.5, 47.25], but against frozen CNNs only +3.83 [1.92, 5.83]. In free-for-all against RuleBots on a fresh block (seed 59000; 180 and 160 games), S3 reached 82.8 [76.1, 88.9] ActiveWR with three agents on 9×9, +46.7 [36.1, 56.7] over G on the same boards, although it was distilled from 1v1 games only; with four agents on 11×11, S3-11 reached 59.4 [50.6, 68.1] (+41.9 over G). Against fair shares of 33% and 25%, 94% and 93% of the students' wins are active, against 64% and 58% for G: opponents also die by their own bombs,

TABLE III

Ablations and controlled negative results (ActiveWR in pp, paired over the stated block). Gate: $\Delta \geq +5$ and CI lower bound > 0. "val." = validation block, descriptive only. Arms from B ran 200 episodes and were evaluated before sudden death existed (the sudden-death rows used an earlier ring schedule); arms from F ran 3,000 episodes under schedule E.

Group	Change	From	Block	ΔActiveWR [95% CI]	Out	Group	Change	From	Block	ΔActiveWR [95% CI]	Out
RL from B	local attack teacher	B	26000	+1.5 [-4.5, 7.25]	fail	RL from F	plain RL	F	42000	−6.5 [-12.75, 0.0]	fail
	two-bomb teacher	B	26000	0.0 (best = start)	fail		sparse search labels	F	42000	−2.75 [-9.75, 4.0]	fail
	plain RL control	B	32000	+2.25 [-3.25, 8.0]	fail		argmax instead of sampling	F	val.	−9.25 [-16.0, -2.5]	worse
	combat MOVE guidance	B	33000	+5.25 [-1.5, 12.0]	fail		escape-cell cut-off rule	F	val.	−1.75 [-4.25, 0.75]	fail
	advantage normalisation	B	34000	+0.25 [-5.5, 6.0]	fail		group adv. $K = 8$, weight 1	F	44000	+0.25 [-5.75, 6.25]	fail
	GAE ($\lambda = 0.95$)	B	35000	0.0 (best = start)	fail		group adv. $K = 16$, weight 4 (= G)	control	45000	+0.75 [-5.75, 7.25]	fail
	half wall reward	B	36000	+0.5	fail	After distil.	self-play RL vs. CNN pool	S2	val.	83.75 → 65.0 / 57.5	lost
	PPO clip $\epsilon = 0.2$	B	37000	−5.5	fail		global KL anchor $\beta = 1$	S2	val.	84 → 68 (control 76)	fail
Sudden death	train under SD	B	40000	−2.25 [-8.0, 3.5]	fail		search models opp. by own CNN	G+s3	val.	−33.0 [-41, -25]	rejected
	+ ring channel	B	40000	−2.25 [-7.5, 3.25]	fail	Accepted	search depth 1 / 2 / 3	G	46k/47k/49k	+16.5 / +21.0 / +8.25	pass
	mask margin $k = 15$	B	41000	+0.75 [-1.25, 2.75]	fail		distillation rounds 1 / 2 / 3	F/G/S2	43k/48k/57k	+16.25 / +35.0 / +10.5	pass

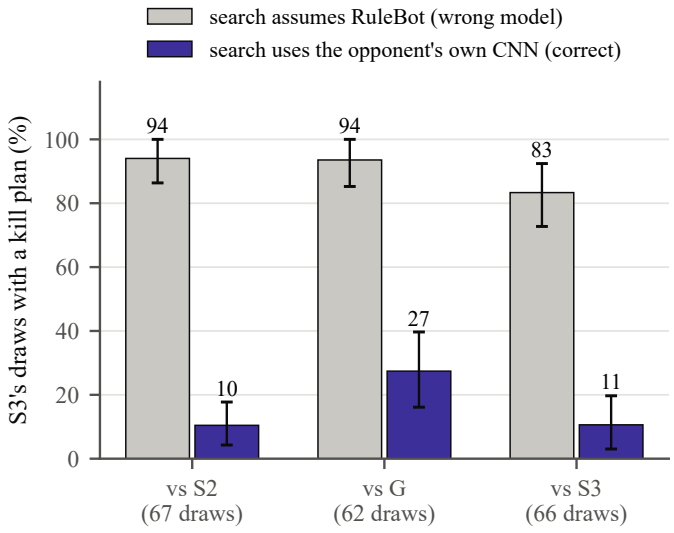


Fig. 7. Share of S3's draws against each CNN in which the depth-3 search finds a kill plan, under the wrong (RuleBot) and the correct (opponent's own CNN) opponent model. Error bars: 95% bootstrap CIs over games.

by each other or in the ring, so WR flatters weak agents most. G+s3 reached 94.4 and 86.3 on an earlier block, so the gap to the search grows outside 1v1.

VII. Discussion

Why dense distillation works where RL does not. Decisive states are rare, and Monte-Carlo policy gradient spreads credit over all steps [28]. A sparse label used once is too little signal; the same labels stored and replayed for several epochs, with counter-examples where BOMB is wrong, teach a conditional rule that RL's one-sided signal could not separate. This matches the finding that more demonstrations matter more than better ones [18]. RL after distillation forgets because our anchors used fixed weights and the KL on decision states was about ten times larger than on ordinary ones; kickstarting needed a decaying weight [22], and imitation-then-PPO in Pommerman also forgot [11].

The ceiling and generality. The search never uses the network, so expert iteration lacks ExIt's step where the apprentice improves the expert [15]; S3 approaches but does not pass 94%. All agents were tested in one environment against one scripted opponent that the search models. The skill survives free-for-all and, after one more distillation round, a larger board, but not frozen CNN opponents. The study therefore supports a claim about turning a search that models a known opponent into a fast network, not about strength against arbitrary opponents.

Comparison with published agents. No agent was run in both environments, so Table IV positions results rather than ranking them. Pommerman is the broader benchmark, but three differences make our numbers conservative. The criterion is stricter: published results count all wins; under that criterion S3 would report 98.0% instead of 92.25%. Waiting is not rewarded: bombs explode after 8 steps instead of 10 and a ring closes the arena, whereas in Pommerman most ties come from the 800-step limit [10]. The budget is small: S3 needs one forward pass per move, while the strongest learning agent in Pommerman needed 200 million steps and still wins only 30% against a depth-1 search [6], [7]. Within these caveats the ordering matches the literature: search leads learning, and in four-agent free-for-all the network-only S3-11 (63.8%) lies within the range of MCTS with learned models (61–78%) [10]. RuleBot and SimpleAgent are both weak heuristics below the fair share (11.9% and about 20% [3]) but were never played against each other.

VIII. Limitations

(1) Only distillation round 3 was replicated over training seeds (three seeds, all pass); other arms used one seed, so their intervals measure variation over boards, not training runs [25], [26]. (2) The reference opponent is RuleBot, and the search assumes RuleBot. (3) Milestones in Table I come from different test blocks. (4) Policies are sampled, so numbers reproduce bit-for-bit only with the same torch version. (5) Free-for-all and 11×11 results are single runs. (6) The agents were not run in the official Pommerman environment or against SimpleAgent.

IX. Conclusion

In Bomberman, win rate rewards survival and opponent mistakes; ActiveWR measures an agent that wins by killing. Measured this way, model-free RL with a safety

TABLE IV
Reported results of published Pommerman agents and of ours. Settings differ (Pommerman: 11×11, 10-step bombs, kicking, 800-step games, partial observation in team mode; ours: no kicking, 8-step bombs, 500 steps with sudden death, full observation), so rows position results and are not a head-to-head comparison. No published work reports ActiveWR.

Work	Setting	Opponents	Method	Reported result
Gao 2019 [4]	FFA / team	3 SimpleAgents / team	PPO + action pruning	WR 40–43% (FFA), 65–76% (team)
Perez-Liebana 2019 [13]	FFA, full observation	3 rule-based agents	MCTS / RHEA	WR 46.5% / 33.0%
Weil 2023 [10]	official FFA	3 SimpleAgents	MCTS + learned model	WR 61–78%, ties 12–34%
Meisheri 2019 [11]	team 2×2, partial	SimpleAgent team	imitation $\rightarrow$ PPO	WR 77.8%
Osogami 2019 [7]	team 2×2	SimpleAgent team	depth-1 search, pessimistic	WR 92.6–99.7%
Huynh 2024 [6]	team 2×2, partial	SimpleAgent; search agent	PPO + LSTM, PBT, 200M steps	WR 98.85%; 30.0% vs search
Ours, G+s3	FFA 11×11, 4 agents	3 RuleBots	G + network-free kill-search	WR 88.8%, ActiveWR 86.3%
Ours, S3-11	FFA 11×11, 4 agents	3 RuleBots	distilled CNN, no search	WR 63.8%, ActiveWR 59.4%
Ours, S3	1v1 9×9	RuleBot	distilled CNN, no search	WR 98.0%, ActiveWR 92.25%

mask stalls below about 50%, a short network-free kill-search reaches 94%, and distilling its labels three times gives a single CNN at 92.25% that needs one forward pass per move. Against equally careful neural opponents most games remain draws, mainly because of the game's rules.

Statements

Data and code. Code, run registrations, per-game logs and figure scripts: anonymised repository https://anonymous.4open.science/r/ANONYMOUS-ID. Supplementary material gives the observation planes, the game-length distributions and the RL training curves. Use of AI tools. An AI assistant helped draft and edit the text and write plotting code; the authors designed and ran the experiments, checked every number against the logs and take responsibility for the content.

References

[1] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis, "Human-level control through deep reinforcement learning," Nature, vol. 518, pp. 529–533, 2015.

[2] O. Vinyals et al., "Grandmaster level in StarCraft II using multi-agent reinforcement learning," Nature, vol. 575, pp. 350–354, 2019.

[3] C. Resnick, W. Eldridge, D. Ha, D. Britz, J. Foerster, J. Togelius, K. Cho, and J. Bruna, "Pommerman: A multi-agent playground," arXiv preprint arXiv:1809.07124, 2018.

[4] C. Gao, B. Kartal, P. Hernandez-Leal, and M. E. Taylor, "On hard exploration for reinforcement learning: A case study in Pommerman," in Proc. AAAI Conf. on Artificial Intelligence and Interactive Digital Entertainment (AIIDE), 2019, arXiv:1907.11788.

[5] P. Peng, L. Pang, Y. Yuan, and C. Gao, "Continual match based training in Pommerman: Technical report," arXiv preprint arXiv:1812.07297, 2018.

[6] N.-M. Huynh, H.-G. Cao, and I.-C. Wu, "Multi-agent training for Pommerman: Curriculum learning and population-based self-play approach," arXiv preprint arXiv:2407.00662, 2024.

[7] T. Osogami and T. Takahashi, "Real-time tree search with pessimistic scenarios: Winning the NeurIPS 2018 Pommerman competition," in Proc. Asian Conf. on Machine Learning (ACML), ser. PMLR, vol. 101, 2019, arXiv:1902.10870.

[8] C. Gao, P. Hernandez-Leal, B. Kartal, and M. E. Taylor, "Skynet: A top deep RL agent in the inaugural Pommerman team competition," in Proc. Multidisciplinary Conf. on Reinforcement Learning and Decision Making (RLDM), 2019, arXiv:1905.01360.

[9] B. Kartal, P. Hernandez-Leal, and M. E. Taylor, "Using Monte Carlo tree search as a demonstrator within asynchronous deep RL," arXiv preprint arXiv:1812.00045, 2018.

[10] J. Weil, J. Czech, T. Meuser, and K. Kersting, "Know your enemy: Investigating Monte-Carlo tree search with opponent models in Pommerman," in Adaptive and Learning Agents Workshop at AAMAS (ALA), 2023, arXiv:2305.13206.

[11] H. Meisheri, O. Shelke, R. Verma, and H. Khadilkar, "Accelerating training in Pommerman with imitation and reinforcement learning," arXiv preprint arXiv:1911.04947, 2019.

[12] C. Resnick, R. Raileanu, S. Kapoor, A. Peysakhovich, K. Cho, and J. Bruna, "Backplay: "Man muss immer umkehren"," arXiv preprint arXiv:1807.06919, 2018.

[13] D. Perez-Liebana, R. D. Gaina, O. Drageset, E. İlhan, M. Balla, and S. M. Lucas, "Analysis of statistical forward planning methods in Pommerman," in Proc. AAAI Conf. on Artificial Intelligence and Interactive Digital Entertainment (AIIDE), vol. 15, no. 1, 2019, pp. 66–72.

[14] X. Guo, S. Singh, H. Lee, R. L. Lewis, and X. Wang, "Deep learning for real-time Atari game play using offline Monte-Carlo tree search planning," in Advances in Neural Information Processing Systems (NIPS), vol. 27, 2014, pp. 3338–3346.

[15] T. Anthony, Z. Tian, and D. Barber, "Thinking fast and slow with deep learning and tree search," in Advances in Neural Information Processing Systems (NIPS), 2017, arXiv:1705.08439.

[16] D. Silver, J. Schrittwieser, K. Simonyan, I. Antonoglou, A. Huang, A. Guez, T. Hubert, L. Baker, M. Lai, A. Bolton, Y. Chen, T. Lillicrap, F. Hui, L. Sifre, G. van den Driessche, T. Graepel, and D. Hassabis, "Mastering the game of Go without human knowledge," Nature, vol. 550, pp. 354–359, 2017.

[17] D. Silver, T. Hubert, J. Schrittwieser, I. Antonoglou, M. Lai, A. Guez, M. Lanctot, L. Sifre, D. Kumaran, T. Graepel, T. Lillicrap, K. Simonyan, and D. Hassabis, "Mastering chess and shogi by self-play with a general reinforcement learning algorithm," arXiv preprint arXiv:1712.01815, 2017.

[18] B. Kartal, P. Hernandez-Leal, C. Gao, and M. E. Taylor, "Safer deep RL with shallow MCTS: A case study in Pommerman," arXiv preprint arXiv:1904.05759, 2019.

[19] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," arXiv preprint arXiv:1503.02531, 2015.

[20] A. A. Rusu, S. G. Colmenarejo, Ç. Gülçehre, G. Desjardins, J. Kirkpatrick, R. Pascanu, V. Mnih, K. Kavukcuoglu, and R. Hadsell, "Policy distillation," in Int. Conf. on Learning Representations (ICLR), 2016, arXiv:1511.06295.

[21] S. Ross, G. J. Gordon, and J. A. Bagnell, "A reduction of imitation learning and structured prediction to no-regret online

learning," in Proc. Int. Conf. on Artificial Intelligence and Statistics (AISTATS), 2011, arXiv:1011.0686.
[22] S. Schmitt, J. J. Hudson, A. Zidek, S. Osindero, C. Doersch, W. M. Czarnecki, J. Z. Leibo, H. Kuttler, A. Zisserman, K. Simonyan, and S. M. A. Eslami, "Kickstarting deep reinforcement learning," arXiv preprint arXiv:1803.03835, 2018.
[23] S. Huang and S. Ontañón, "A closer look at invalid action masking in policy gradient algorithms," in Proc. Int. FLAIRS Conference, vol. 35, 2022, arXiv:2006.14171.
[24] A. Y. Ng, D. Harada, and S. Russell, "Policy invariance under reward transformations: Theory and application to reward shaping," in Proc. Int. Conf. on Machine Learning (ICML), 1999, pp. 278–287.
[25] P. Henderson, R. Islam, P. Bachman, J. Pineau, D. Precup, and D. Meger, "Deep reinforcement learning that matters," in Proc. AAAI Conf. on Artificial Intelligence, 2018, arXiv:1709.06560.
[26] R. Agarwal, M. Schwarzer, P. S. Castro, A. Courville, and M. G. Bellemare, "Deep reinforcement learning at the edge of the statistical precipice," in Advances in Neural Information Processing Systems (NeurIPS), 2021, arXiv:2108.13264.
[27] B. Efron, "Bootstrap methods: Another look at the jackknife," The Annals of Statistics, vol. 7, no. 1, pp. 1–26, 1979.
[28] R. J. Williams, "Simple statistical gradient-following algorithms for connectionist reinforcement learning," Machine Learning, vol. 8, pp. 229–256, 1992.
[29] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, "High-dimensional continuous control using generalized advantage estimation," in Int. Conf. on Learning Representations (ICLR), 2016, arXiv:1506.02438.
[30] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," arXiv preprint arXiv:1707.06347, 2017.
[31] Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, X. Bi, H. Zhang, M. Zhang, Y. K. Li, Y. Wu, and D. Guo, "DeepSeekMath: Pushing the limits of mathematical reasoning in open language models," arXiv preprint arXiv:2402.03300, 2024.